

eman ta zabal zazu



Universidad  
del País Vasco

Euskal Herriko  
Unibertsitatea

Ingeniería Informática de Gestión y Sistemas de  
Información

Desarrollo Avanzado de Software

**RedJack**

Autor:

Luken Bilbao Rojo

21 de abril de 2025

# Índice

|                                             |           |
|---------------------------------------------|-----------|
| <b>1. Introducción</b>                      | <b>2</b>  |
| <b>2. Información general</b>               | <b>2</b>  |
| <b>3. Objetivos alcanzables</b>             | <b>2</b>  |
| 3.1. Objetivos obligatorios . . . . .       | 3         |
| 3.2. Objetivos adicionales . . . . .        | 4         |
| <b>4. Diseño</b>                            | <b>4</b>  |
| 4.1. RegisterActivity.java . . . . .        | 4         |
| 4.2. conexionBDWebService.java . . . . .    | 5         |
| 4.3. MapActivity.java . . . . .             | 5         |
| 4.4. Cambios en MainActivity.java . . . . . | 6         |
| 4.5. ReminderReceiver.java . . . . .        | 7         |
| 4.6. ProfileWidget.java . . . . .           | 7         |
| 4.7. api.php . . . . .                      | 7         |
| 4.8. monedas.php . . . . .                  | 7         |
| 4.9. pfp.php . . . . .                      | 7         |
| <b>5. Manual de uso</b>                     | <b>8</b>  |
| <b>6. Dificultades</b>                      | <b>18</b> |
| <b>7. Fuentes</b>                           | <b>18</b> |

## 1. Introducción

RedJack es una aplicación para dispositivos móviles desarrollada mediante el framework de desarrollo nativo Android Studio. El objetivo del programa desarrollado es ser entretenimiento para el usuario, el cual podrá jugar a una variante del clásico juego de cartas Blackjack.

Además de esto, este proyecto tiene como objetivo evaluar las capacidades de su autor para poner en práctica técnicas y métodos relativos a Android Studio vistos en clase, a fin de lograr una aplicación funcional, eficiente e intuitiva.

Cabe destacar que la aplicación está desarrollada en Java en su totalidad, utilizando Android Studio y siguiendo además las mejores prácticas de desarrollo para Android.

Nótese que este es el segundo de dos informes relativos a este proyecto, el cual hace referencia a los objetivos de la segunda entrega del mismo.

## 2. Información general

En esta breve sección se expondrá información relevante en lo que a pruebas se refiere.

- **GitHub de la aplicación:** <https://github.com/cursedx0/DAS-RedJack> (segunda entrega en branch 'entrega2').
- **Usuario de pruebas:** xxx
- **Contraseña de pruebas:** xxx

## 3. Objetivos alcanzables

En esta sección se resumirán los objetivos alcanzados y su papel dentro del contexto de la aplicación.

### 3.1. Objetivos obligatorios

|                                                                                                                            |                                                                                                                                                           |
|----------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Uso de una base de datos remota para el registro y la identificación de usuarios mediante registro.                        | Realizado. Los usuarios pueden registrarse en una pantalla de registro para posteriormente iniciar sesión en la pantalla inicial.                         |
| Integrar los servicios Google Maps u Open Street Map y Geolocalización en una actividad.                                   | Realizado. La actividad MapActivity muestra los casinos y salones de juego cercanos en el radio establecido por el usuario.                               |
| Captar imágenes desde la cámara, guardarlas en el servidor y mostrarlas en la aplicación. Por ejemplo, una foto de perfil. | Realizado. En la actividad ProfileActivity, el usuario puede ver y modificar su foto de perfil, ya sea usando la cámara o eligiendo una desde la galería. |

Cuadro 1: Objetivos obligatorios del proyecto.

### 3.2. Objetivos adicionales

|  |                                                                                                                                                                               |                                                                                                                                                              |
|--|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | Uso de algún content provider para añadir, modificar o eliminar datos.                                                                                                        | No realizado.                                                                                                                                                |
|  | Implementación de un servicio en primer plano y gestión de mensajes broadcast durante el servicio.                                                                            | No realizado.                                                                                                                                                |
|  | Uso de mensajería FCM. Se debe incluir alguna forma de que se pueda probar de forma externa (por ejemplo, con un servicio web PHP adicional en el servidor de la asignatura). | No realizado.                                                                                                                                                |
|  | Desarrollar un widget que tenga, al menos, un elemento que se actualice automáticamente de manera periódica                                                                   | Realizado. El usuario puede establecer un widget en la pantalla de inicio, donde se muestra información sobre su perfil (victorias totales, saldo...).       |
|  | Uso de algún servicio o tarea programada mediante alarma (no valen las alarmas del widget).                                                                                   | Realizado. Cuando el usuario abandona la aplicación y no entra en cierto período de tiempo, se le manda periódicamente una notificación invitándolo a jugar. |

Cuadro 2: Objetivos adicionales del proyecto.

## 4. Diseño

En esta sección se describirán las clases de Java añadidas en esta segunda entrega más relevantes para el correcto funcionamiento de la aplicación y sus atributos y métodos más importantes, así como explicaciones sobre la lógica interna de algunos componentes.

### 4.1. RegisterActivity.java

Actividad en la que el usuario podrá registrar una cuenta nueva. De ya tener una, podrá iniciar sesión desde la pantalla principal sin problema.

Su lógica es similar a la del inicio de sesión, salvo que la operación solicitada al servidor es la de registrar un nuevo usuario, no verificar su existencia.

Esta actividad no contiene atributos ni métodos dignos de mención.

## 4.2. conexionBDWebService.java

Esta clase que extiende de la clase Worker de Java, será la encargada de comunicar al servidor web qué operación debe hacerse y con qué parámetros.

Toda su lógica está contenida en la sobreescritura del método doWork(), el cual nos permite realizar las siguientes operaciones de cara al servidor:

- **insertar:** Operación para insertar un nuevo usuario en la base de datos. Usada en la pantalla de registro.
- **login:** Operación para verificar la existencia de un usuario mediante su nombre de usuario y su contraseña. Usada en la pantalla de inicio de sesión.
- **eliminar:** Operación para eliminar un usuario. No implementada.
- **info:** Operación que recupera de la base de datos información sobre un nombre de usuario dado. Obtiene información como el saldo, el número de victorias... Usada en la pantalla de perfil y widget.
- **monedas:** Operación para obtener el número de monedas de un usuario dado. Usado en la pantalla de juego.
- **sumar:** Operación para añadir monedas a un usuario. Al mismo tiempo, esta función se encarga de añadir victorias o empates al usuario. Internamente, si se le pasa un parámetro 'empate' no vacío, se le sumará un empate en vez de una victoria. Independientemente de si se trata de una victoria o empate o victoria, se resta una derrota al usuario. Esto se razona más adelante. Usada en pantalla de juego.
- **restar:** Operación para restar monedas al jugador. También se encarga de sumar derrotas. Esta función se ejecuta siempre al hacerla apuesta, de manera que aunque el jugador salga de la partida, pierda las monedas y se le cuente como derrota. Por esto mismo, la función 'sumar' debe restar una derrota al terminar la partida (si es que el jugador no pierde). Usada en la pantalla de juego.
- **setpfp:** Operación para establecer una nueva imagen de perfil (ProFile Picture). Utilizada en la pantalla de de perfil.
- **getpfp:** Operación para obtener la imagen de perfil del servidor. Usada en la pantalla de perfil.

## 4.3. MapActivity.java

Esta actividad tiene como objetivo albergar un mapa que, situando al usuario en este, mostrará la ubicación de casinos y salones de juego cercanos, en el radio establecido a través de un input.

Cabe destacar que los métodos `onPause()`, `onStop()` y `onDestroy()` han sido sobrescritos para poder manejar la salida y vuelta a la actividad del usuario, evitando null pointers y problemas derivados.

Estos serán los métodos que implementará:

- **protected void onCreate():** método `onCreate`. Establece los `onClickListeners` y llama a la función de solicitud de permisos.
- **private void requestLocationPermission():** método encargado de pedir los permisos de ubicación en caso de no haber sido concedidos. De estar concedidos o concederse en el momento, se lanzaría el método que empezaría las actualizaciones de ubicación.
- **private void startLocationUpdates():** método llamado al empezar la localización. Esconde botones y textos y comienza la escucha de la ubicación con el `LocationListener` y `LocationManager`.
- **private void rutinaUbicacion():** método llamado al actualizar manual o automáticamente (por `locationUpdate`) el mapa. Llama a `updateMapLocation()` para actualizar la posición del usuario y a `obtenerCasinos()` para actualizar la posición de los casinos. Los casinos del mapa pueden aparecer y desaparecer en función de las distancias establecidas al ejecutarse este método.
- **private void updateMapLocation():** método utilizado para actualizar la posición del usuario en el mapa. Al tener que limpiar el mapa, se ha visto la necesidad de llamar a `obtenerCasinos()` dentro de este método para que no desaparecieran.
- **private void obtenerCasinos():** método que localiza y señala los casinos en el mapa. La query contenida en este método dependerá de la distancia establecida

#### 4.4. Cambios en MainActivity.java

Esta actividad, que alberga la pantalla de inicio de sesión de la aplicación, ha sufrido algunos cambios importantes que se describirán más abajo.

Como base, se ha migrado el uso de la base de datos de local a remoto. Esto también se ha hecho en la actividad de juego (`PlayActivity`). Sin embargo, el cambio más destacable en esta actividad es la integración de la programación de una alarma.

Los siguientes dos métodos se utilizan para programar notificaciones periódicas tras la salida del usuario de la app, haciendo uso de la clase `ReminderReceiver` (que extiende de `BroadcastReceiver`):

- **private void programarNotificacionEnUnaHora()**: método que programa una alarma para dentro de media hora (a pesar de lo que su nombre pueda sugerir). Dado que no se utiliza una alarma exacta (porque su objetivo no requiere de ese nivel de precisión), se vio más oportuno utilizar 30 minutos como periodo de notificación suficiente. Esta función se ejecutará en el método `onStop()`, de manera que se programará cuando el usuario abandone la aplicación.
- **private void cancelarNotificacionProgramada()**: método utilizado para revertir las acciones del anterior, cancelando la alarma programada. Será llamado en el método `onStart()` para evitar este tipo de notificaciones dentro de la app.

#### 4.5. **ReminderReceiver.java**

Clase encargada de gestionar las acciones asociadas a las alarmas establecidas, la cual extiende de `BroadcastReceiver`. Su única función es escuchar el momento en el que una alarma se activa para llevar acabo ciertas acciones, que en esta caso es la muestra de una notificación que pide al usuario que vuelva a jugar.

#### 4.6. **ProfileWidget.java**

Clase encargada de gestionar la lógica del widget de estadísticas del usuario. Implementa el método `public void onUpdate()`, el cual es encarga de la tareas a la hora de actualizar el widget.

#### 4.7. **api.php**

Archivo `.php` en el servidor encargado de gestionar peticiones a la base de datos relacionadas con la gestión de usuarios. Este archivo `.php`, así como los explayados a continuación, son llamados por la clase `conexionBDWebService` para atender las peticiones de la aplicación.

#### 4.8. **monedas.php**

Archivo `.php` en el servidor encargado de gestionar peticiones a la base de datos relacionadas con la gestión de monedas, victorias, derrotas y empates de los usuarios.

#### 4.9. **pfp.php**

Archivo `.php` en el servidor encargado de gestionar peticiones a la base de datos relacionadas con la gestión de las imágenes de perfil de los usuarios. Estas son almacenadas en la base de datos, así como en la carpeta `'pfps'` del servidor. En un principio se intentó obtener la imagen de la base de datos, pero a pesar de aplicar compresiones y reescalados, la imagen resultaba demasiado pesada



para su manejo, por lo que se optó por usar la librería Glide para solicitarla al servidor.

## 5. Manual de uso

En esta sección se explicará el funcionamiento y flujo habitual de las nuevas funcionalidades de la aplicación con algunas capturas de pantalla de esta.

1. Al entrar en la aplicación, nos encontramos con una pantalla para iniciar sesión. Las credenciales de prueba se encuentran en la sección “Información general”, pero en este caso podremos crear nuestro propio usuario si gustamos, pulsando el botón en la parte inferior de la pantalla..

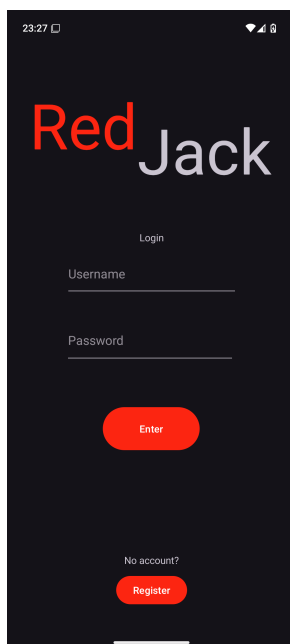


Figura 1: MainActivity.java. El tema e idioma por defecto pueden diferir de los mostrados en las imágenes.

2. Este botón nos llevará a la pantalla de registro, la cual nos pide un nombre de usuario y una contraseña para este. Si introducimos un usuario que no esté en uso y una contraseña cualquiera, nuestro usuario será creado y guardado en la base de datos. En caso de que nuestro nombre de usuario esté cogido, la aplicación nos avisará de ello.

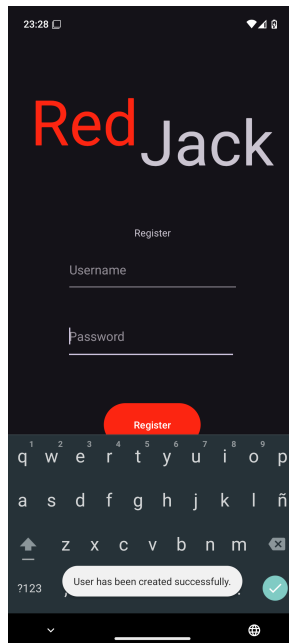


Figura 2: RegisterActivity.java. Se nos informará del resultado del registro con un mensaje Toast.

3. Una vez registrados, podremos regresar a la pantalla de inicio de sesión para entrar con nuestro usuario recién creado. Una vez dentro, podremos abrir el menú de la toolbar, que mostrará dos nuevos ítems.

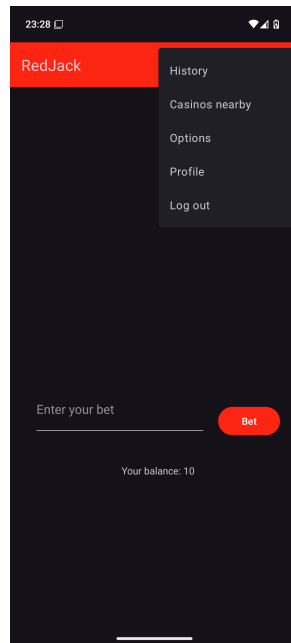


Figura 3: Menú oculto expandido.

- a) El primer ítem nuevo del menú es el mapa de casinos cercanos. Al entrar, si no hemos concedido los permisos antes, nos pedirá permisos de ubicación precisa.



Figura 4: MapActivity pidiendo permisos.

- b) En caso de no conceder los permisos o solamente conceder los permisos de ubicación aproximada, el mapa no se mostrará. Podremos cambiar los permisos pulsando el botón de 'Pregúntame otra vez', que nos llevará a los ajustes de la aplicación en el teléfono.

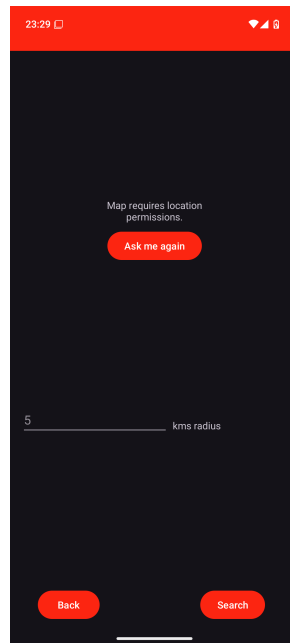


Figura 5: MapActivity sin permisos.

- c) Al concederse los permisos, el mapa señalará con un marcador verde nuestra posición actual. Por defecto, se mostrarán los casinos y salones de juego en 5 kilómetros a la redonda. Dependiendo de nuestra posición, quizás debamos aumentar el rango para que aparezcan resultados.

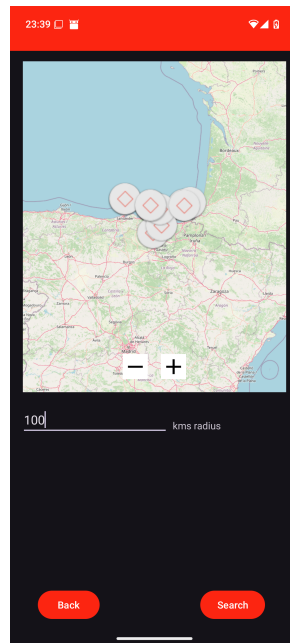


Figura 6: Casinos cercanos. Podemos cambiar el rango de búsqueda dinámicamente esperando a que se actualice la posición o forzando la búsqueda con el botón en la esquina inferior derecha.

- a) El segundo y último ítem nuevo es el del perfil. Cuando entremos por primera vez, se nos mostrará nuestro nombre de usuario, estadísticas y una imagen de perfil por defecto.

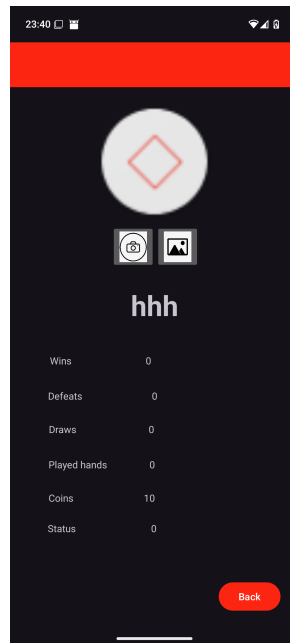


Figura 7: Perfil por defecto de jugador 'hhh'.

- b) Para cambiarla, podremos pulsar los botones de cámara y galería, los cuales nos permitirán sacar una foto o elegir una de nuestro dispositivo.

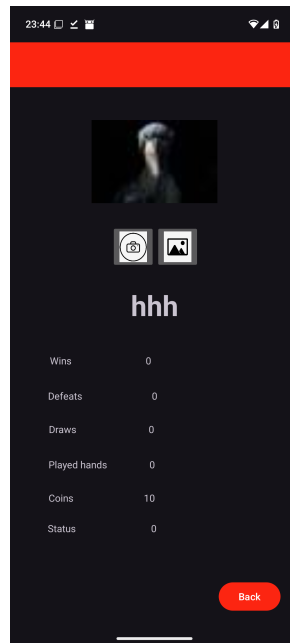


Figura 8: Perfil con foto de perfil cambiada. El cambio puede tardar unos segundos en surtir efecto.

4. Además de estas dos funcionalidades, esta entrega incluye la adición de un widget de la aplicación para la pantalla de inicio. Para utilizarlo, mantémoslo pulsado en un hueco vacío de la pantalla de inicio de nuestro teléfono y pulsamos la opción 'Widgets'.



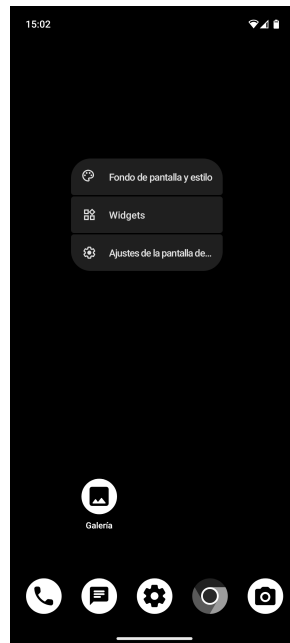


Figura 9: Tras mantener pulsado un hueco vacío, aparecerán estas opciones. Deberemos pulsar 'Widgets'.

5. Después, elegiremos la aplicación RedJack, mantendremos pulsado el único widget que contiene y lo arrastraremos a algún hueco en la pantalla de inicio.

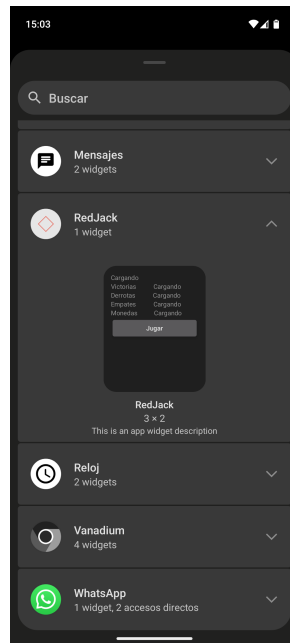


Figura 10: Widgets de las aplicaciones.



Figura 11: Widget una vez arrastrado a la pantalla de inicio. Las estadísticas se actualizan periódicamente y el botón 'Jugar' abre la aplicación.

## 6. Dificultades

A lo largo del proyecto, me he encontrado con algunas dificultades en lo que a desarrollo y diseño se refiere. Estas serán listadas en esta sección.

- La representación del mapa. Debido al cambio de OSM por Google Maps, algunas partes del desarrollo usaban librerías de este que colisionaban con las de OSM y llevó cierto tiempo ordenar el diseño.
- Las imágenes de perfil. A la hora de transportar la imagen del teléfono a la base de datos y servidor no hubo mayor problema, pero cuando se trataba de recuperarla para mostrarla en la app, la imagen suponía una carga demasiado grande como para descargarla con métodos convencionales. Al final opté por guardarlas en el servidor y usar la librería Glide para obtenerlas de ahí.

## 7. Fuentes

- **Android Studio Documentation:** <https://developer.android.com/develop>

- **Apuntes de eGela:** <https://egela.ehu.eus>
- **Overleaf Documentation:** <https://www.overleaf.com/learn>
- **ChatGPT:** [chatgpt.com](https://chatgpt.com)
- **OSM Map does not display** <https://stackoverflow.com/questions/43169919/why-map-in-osm-not-display-in-android-studio>
- **OSM Documentation** <https://wiki.openstreetmap.org/>
- **PHP Documentation** <https://www.php.net/docs.php>
- **PHP file\_put\_contents()** [https://www.w3schools.com/php/func\\_filesystem\\_file\\_put\\_contents.asp](https://www.w3schools.com/php/func_filesystem_file_put_contents.asp)
- **Glide Example** <https://www.geeksforgeeks.org/image-loading-caching-library-android-set-2/>
- **Glide Repository** <https://github.com/bumptech/glide>
- **MySQL Documentation** <https://dev.mysql.com/doc/>